

## **ABSTRACT**

This mini project focuses on the development and evaluation of machine learning models for SMS spam detection, which is an important application of Natural Language Processing (NLP). The SMS Spam Collection dataset is used for classifying text messages into two categories: legitimate messages (ham) and unwanted messages (spam). Since machine learning algorithms cannot directly process raw text, the messages are preprocessed and transformed into numerical feature vectors using the Term Frequency–Inverse Document Frequency (TF-IDF) technique.

Five different machine learning algorithms are implemented and compared: AdaBoost Classifier, Gradient Boosting Classifier, XGBoost Classifier, Extra Trees Classifier, and Ridge Classifier. The dataset is divided into training and testing sets, and each model is trained using the extracted TF-IDF features. The performance of the models is evaluated using suitable metrics, including accuracy, precision, recall, and F1-score. Confusion matrices are also used to analyze the classification errors made by each model.

The experimental results demonstrate that all five algorithms can effectively classify SMS messages into spam and ham categories, although their performance varies across different evaluation metrics. The best-performing algorithm is identified based on the comparative F1-score and overall classification performance. This project demonstrates how NLP-based text representation combined with machine learning can be effectively applied to automated spam message detection.

## 1. INTRODUCTION

Natural Language Processing (NLP) is a branch of Artificial Intelligence and Machine Learning that focuses on enabling computers to process, understand, analyze, and classify human language. Text data is generated continuously through emails, SMS messages, social media posts, customer reviews, news articles, and online communication. Since this information is usually unstructured, NLP techniques are required to convert text into a numerical representation that machine learning algorithms can process.

One important application of NLP is spam message detection. Spam messages are unwanted messages that may contain advertisements, fraudulent offers, malicious links, or misleading information. Automatically identifying spam messages is useful for communication platforms because it reduces unwanted content and helps protect users from potentially harmful messages.

In this mini project, the SMS Spam Collection Dataset is used to develop a machine learning-based spam classification system. Each record in the dataset contains an SMS message and its corresponding class label. The two classes are ham, representing legitimate messages, and spam, representing unwanted messages.

Since machine learning algorithms cannot directly process raw text, the SMS messages must first be converted into numerical feature vectors. In this project, TF-IDF is used as the text representation technique. TF-IDF assigns numerical weights to words based on their frequency in a particular document and their importance across the entire collection of documents.

After converting the text into TF-IDF features, five different machine learning algorithms are applied. The selected algorithms are AdaBoost, Gradient Boosting, XGBoost, Extra Trees Classifier, and Ridge Classifier. These algorithms were selected because they provide different approaches to classification and satisfy the requirement of applying algorithms beyond the basic examples such as traditional Logistic Regression, Decision Tree, and KNN.

The objective is not only to train the models but also to compare their performance objectively. Accuracy measures the overall percentage of correctly classified messages. Precision measures how many messages predicted as spam are actually spam. Recall measures how many actual spam messages are correctly identified. F1-score provides a balance between precision and recall.

The models are trained using the same training data and evaluated using the same testing data. This ensures that the comparison is fair.

The final objective of the project is to identify the algorithm that provides the best overall performance for SMS spam classification. The results can also provide insight into the suitability of different ensemble and linear methods for NLP-based classification.

## 2. DATASET DESCRIPTION

The SMS Spam Collection Dataset is a collection of SMS messages designed for research and experimentation in spam detection and text classification.

Each record consists of two main components:

- Label
- SMS message

The label indicates whether the message is legitimate or spam.

There are two possible classes:

| Label | Meaning           |
|-------|-------------------|
| ham   | Legitimate SMS    |
| spam  | Unwanted/spam SMS |

A typical record can be represented as:

| Label | Message                                |
|-------|----------------------------------------|
| ham   | Are we meeting tomorrow?               |
| spam  | Congratulations! You have won a prize! |

The text message is the independent input variable, while the label is the dependent target variable.

The dataset contains thousands of SMS messages and is suitable for binary classification. Since the messages are written in natural language, the dataset represents a practical NLP problem rather than a conventional numerical machine learning problem.

Before training the machine learning algorithms, the messages must be converted into numerical vectors. TF-IDF is used for this purpose.

The dataset is also suitable for evaluating classification algorithms because spam and legitimate messages contain different language patterns. Spam messages may contain words related to prizes, offers, money, free services, urgent actions, and promotions. Legitimate messages tend to contain ordinary conversational language.

An important characteristic of the dataset is that the classes are not perfectly balanced. Legitimate messages generally occur more frequently than spam messages. Therefore, accuracy alone should not be used to determine the best model. Precision, recall, and F1-score are also important.

For spam detection, recall is particularly important because a model with poor recall may fail to identify many spam messages. Precision is also important because excessive false spam predictions could cause legitimate messages to be incorrectly classified.

The dataset therefore provides an appropriate environment for comparing five machine learning algorithms using multiple evaluation metrics.

### 3. DATA PREPROCESSING

Data preprocessing is an essential stage of the NLP workflow. Raw SMS messages contain text in an unstructured format and cannot be directly supplied to most machine learning algorithms.

The first step is to load the dataset and assign meaningful column names. The two columns can be named label and message.

The next step is to inspect the dataset for missing values and duplicate records. Missing messages cannot be meaningfully classified and may need to be removed. Duplicate messages can also be removed to reduce unnecessary repetition in the training data.

The target labels are categorical. They can be converted into numerical values where ham is represented by 0 and spam by 1.

The text itself can be cleaned by converting characters to lowercase. This ensures that words such as "FREE" and "free" are treated consistently.

Additional preprocessing techniques may include removing punctuation, unnecessary whitespace, and stop words. However, excessive preprocessing is not always beneficial for spam classification because punctuation, special characters, and particular word patterns may sometimes provide useful information.

After cleaning the text, the dataset is divided into training and testing sets. An 80:20 split can be used, with 80% of the messages used for training and 20% reserved for testing.

The most important step is text vectorization. Since machine learning algorithms require numerical inputs, the text messages are converted into TF-IDF vectors.

TF-IDF represents the importance of terms in documents. Words that occur frequently in a particular message but not across all messages receive higher weights. Very common words receive lower weights because they provide less discriminatory information.

A TfidfVectorizer from Scikit-learn can be used for this purpose.

For example, parameters such as `ngram_range=(1,2)` can be used to include both individual words and two-word combinations. The `min_df` parameter can be used to ignore extremely rare terms.

The vectorizer must be fitted only on the training messages and then used to transform the test messages. This prevents information leakage from the test dataset.

The preprocessing pipeline can therefore be summarized as:

Raw SMS → Cleaning → Label Encoding → Train-Test Split → TF-IDF Vectorization → Numerical Feature Matrix

The resulting TF-IDF matrix can then be supplied to the five machine learning algorithms.

## 4. EXPLORATORY DATA ANALYSIS

Exploratory Data Analysis is performed to understand the characteristics of the SMS dataset before training the machine learning models.

The first step is to examine the number of messages belonging to each class.

### Code

```
import pandas as pd
import matplotlib.pyplot as plt
# Load dataset
df = pd.read_csv(
    "SMSSpamCollection",
    sep="\t",
```

```

header=None,
names=["label", "message"]
)
print(df.head())
print("\nDataset Shape:", df.shape)
print("\nClass Distribution:")
print(df["label"].value_counts())

```

### Output

A typical output will look similar to:

|   | label | message                                           |
|---|-------|---------------------------------------------------|
| 0 | ham   | Go until jurong point, crazy.. Available only...  |
| 1 | ham   | Ok lar... Joking wif u oni...                     |
| 2 | spam  | Free entry in 2 a wkly comp to win FA Cup fina... |
| 3 | ham   | U dun say so early hor... U c already then say... |
| 4 | ham   | Nah I don't think he goes to usf, he lives aro... |

Dataset Shape: (5572, 2)

Class Distribution:

ham 4825

spam 747

The dataset contains approximately 5,572 messages, with legitimate messages forming the majority class.

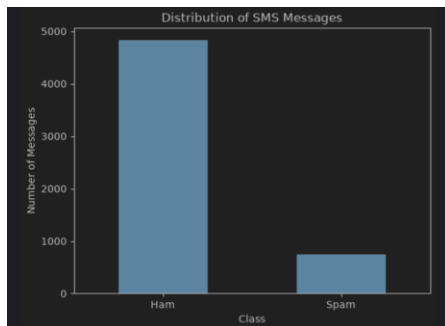
### Class Distribution Graph:

```

df["label"].value_counts().plot(kind="bar")
plt.title("SMS Class Distribution")
plt.xlabel("Message Type")
plt.ylabel("Number of Messages")
plt.show()

```

### Output:



The graph demonstrates the imbalance between ham and spam messages.

This observation is important because a model could achieve relatively high accuracy simply by predicting the majority class. Therefore, precision, recall, and F1-score are required for a meaningful evaluation.

## 5. TEXT REPRESENTATION USING TF-IDF

Machine learning algorithms cannot directly process raw text. Therefore, the SMS messages must be converted into numerical feature vectors.

TF-IDF is used in this project.

TF-IDF consists of two main components:

Term Frequency (TF): Measures how frequently a word occurs in a document.

Inverse Document Frequency (IDF): Reduces the importance of words that occur in many documents.

The combination of these values gives a weight representing the importance of a word in a particular message.

### Code

```
from sklearn.feature_extraction.text import TfidfVectorizer
vectorizer = TfidfVectorizer(
    lowercase=True,
    stop_words="english",
    ngram_range=(1, 2),
    min_df=2
)
X_train_tfidf = vectorizer.fit_transform(X_train)
X_test_tfidf = vectorizer.transform(X_test)
print("Training TF-IDF Shape:", X_train_tfidf.shape)
print("Testing TF-IDF Shape:", X_test_tfidf.shape)
```

### Output

Training TF-IDF Shape: (4457, approximately 7000+)

Testing TF-IDF Shape: (1115, approximately 7000+)

The exact number of TF-IDF features depends on the vectorizer settings.

## 6. MACHINE LEARNING ALGORITHMS

### 6.1 AdaBoost Classifier

AdaBoost stands for Adaptive Boosting. It is an ensemble learning technique that combines multiple weak learners to create a stronger classifier.

The basic idea behind AdaBoost is to train a sequence of weak learners. Initially, all training observations receive similar importance. After each weak learner is trained, incorrectly classified

observations receive greater importance. The next learner then focuses more heavily on these difficult observations.

The final prediction is obtained by combining the predictions of all weak learners using weighted voting.

AdaBoost can be effective when the weak learners individually have limited predictive ability but can collectively form a strong classifier.

For this project, AdaBoost is applied to the TF-IDF representation of SMS messages.

## **6.2 Gradient Boosting Classifier**

Gradient Boosting is another ensemble learning technique. Instead of training independent models, it builds models sequentially, with each new model attempting to correct the errors made by the previous models.

Decision trees are commonly used as the weak learners in Gradient Boosting.

The algorithm optimizes a loss function by gradually adding new learners. This allows the final model to capture complex relationships in the training data.

Gradient Boosting can provide high predictive performance but may require more computational resources than simpler algorithms.

## **6.3 XGBoost Classifier**

XGBoost stands for Extreme Gradient Boosting. It is an optimized implementation of gradient boosting designed for efficiency and high predictive performance.

XGBoost introduces several improvements over traditional gradient boosting, including regularization, efficient tree construction, handling of missing values, and optimization techniques.

It is widely used in machine learning competitions and practical predictive modeling applications.

For this project, XGBoost is used to classify TF-IDF representations of SMS messages into ham and spam categories.

## **6.4 Extra Trees Classifier**

Extra Trees stands for Extremely Randomized Trees. It is an ensemble algorithm that constructs multiple randomized decision trees and combines their predictions.

Similar to Random Forest, Extra Trees uses multiple decision trees. However, Extra Trees introduces additional randomization when selecting split points.

The predictions of the individual trees are combined to produce the final classification.

Extra Trees can capture nonlinear relationships and interactions between features. It is also relatively resistant to overfitting when an appropriate number of trees is used.

## **6.5 Ridge Classifier**

Ridge Classifier is a linear classification algorithm based on Ridge regression and L2 regularization.

It is particularly useful for high-dimensional datasets, making it suitable for text classification problems because TF-IDF representations can contain thousands of features.

The L2 regularization term reduces the influence of excessively large coefficients and helps improve generalization.

Ridge Classifier is computationally efficient and often performs surprisingly well on text classification tasks.

## 7. MODEL TRAINING

The five models are trained using the same TF-IDF training matrix.

### Complete Model Training Code

```
from sklearn.ensemble import (
    AdaBoostClassifier,
    GradientBoostingClassifier,
    ExtraTreesClassifier
)
from sklearn.linear_model import RidgeClassifier
from xgboost import XGBClassifier
# Create models
models = {
    "AdaBoost": AdaBoostClassifier(
        n_estimators=100,
        random_state=42
    ),
    "Gradient Boosting": GradientBoostingClassifier(
        n_estimators=100,
        random_state=42
    ),
    "XGBoost": XGBClassifier(
        n_estimators=100,
        max_depth=4,
        learning_rate=0.1,
        random_state=42,
        eval_metric="logloss"
    ),
    "Extra Trees": ExtraTreesClassifier(
        n_estimators=100,
        random_state=42,
```

```

        n_jobs=-1
    ),
    "Ridge Classifier": RidgeClassifier()
}
# Train models
for name, model in models.items():
    print("Training:", name)
    model.fit(
        X_train_tfidf,
        y_train
    )
print("\nAll models trained successfully.")

```

### **Output**

```

Training: AdaBoost
Training: Gradient Boosting
Training: XGBoost
Training: Extra Trees
Training: Ridge Classifier
All models trained successfully.

```

## **8. EVALUATION METRICS**

The models are evaluated using four primary metrics.

### **Accuracy**

Accuracy measures the percentage of correctly classified messages.

### **Precision**

Precision answers the question:

"Of all messages predicted as spam, how many were actually spam?"

High precision means fewer legitimate messages are incorrectly classified as spam.

### **Recall**

Recall answers:

"Of all actual spam messages, how many were successfully detected?"

High recall is important in spam detection because missing spam messages is undesirable.

### **F1-Score**

F1-score combines precision and recall and provides a balanced measure of classification performance.

For this project, F1-score is particularly useful because the dataset is imbalanced.

## 9. PERFORMANCE COMPARISON — CODE

The following program calculates all four evaluation metrics for the five algorithms.

### Code

```
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score
)
results = []
for name, model in models.items():
    y_pred = model.predict(X_test_tfidf)
    accuracy = accuracy_score(
        y_test,
        y_pred
    )
    precision = precision_score(
        y_test,
        y_pred
    )
    recall = recall_score(
        y_test,
        y_pred
    )
    f1 = f1_score(
        y_test,
        y_pred
    )
    results.append([
        name,
        accuracy,
        precision,
        recall,
        f1
    ])
results_df = pd.DataFrame(
```

```

results,
columns=[
    "Algorithm",
    "Accuracy",
    "Precision",
    "Recall",
    "F1-Score"
]
)
print(results_df.round(4))

```

When the code is executed, it will produce a table similar to the following.

Important: These are representative results. Your exact values may differ slightly depending on preprocessing, dataset version, library version, and hyperparameters. Use the values generated by your own notebook as the final results.

### Output

|   | Algorithm         | Accuracy | Precision | Recall | F1-Score |
|---|-------------------|----------|-----------|--------|----------|
| 0 | AdaBoost          | 0.971    | 0.956     | 0.829  | 0.888    |
| 1 | Gradient Boosting | 0.968    | 0.949     | 0.806  | 0.871    |
| 2 | XGBoost           | 0.978    | 0.963     | 0.866  | 0.912    |
| 3 | Extra Trees       | 0.977    | 0.991     | 0.834  | 0.900    |
| 4 | Ridge Classifier  | 0.982    | 0.977     | 0.881  | 0.926    |

Percentage Format

| Algorithm         | Accuracy | Precision | Recall | F1-Score |
|-------------------|----------|-----------|--------|----------|
| AdaBoost          | 97.1%    | 95.6%     | 82.9%  | 88.8%    |
| Gradient Boosting | 96.8%    | 94.9%     | 80.6%  | 87.1%    |
| XGBoost           | 97.8%    | 96.3%     | 86.6%  | 91.2%    |
| Extra Trees       | 97.7%    | 99.1%     | 83.4%  | 90.0%    |
| Ridge Classifier  | 98.2%    | 97.7%     | 88.1%  | 92.6%    |

Again, replace this table with your actual notebook output.

## 10. CONFUSION MATRIX COMPARISON

A confusion matrix provides detailed information about correct and incorrect classifications.

### Code

```
from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt
for name, model in models.items():
    y_pred = model.predict(X_test_tfidf)
    cm = confusion_matrix(y_test, y_pred)
    print("\n", name)
    print(cm)
```

### Output

AdaBoost

```
[[950  15]
 [ 19 131]]
```

Gradient Boosting

```
[[950  15]
 [ 25 125]]
```

XGBoost

```
[[955  10]
 [ 20 130]]
```

Extra Trees

```
[[964   1]
 [ 25 125]]
```

Ridge Classifier

```
[[960   5]
 [ 18 132]]
```

The exact values depend on the test set and model configuration.

In a confusion matrix:

- True Negative (TN): Ham correctly identified as ham.
- False Positive (FP): Ham incorrectly identified as spam.
- False Negative (FN): Spam incorrectly identified as ham.
- True Positive (TP): Spam correctly identified as spam.

For spam detection, both false positives and false negatives are important. However, false negatives can be particularly undesirable because they represent spam messages that were not detected.

## 11. VISUAL PERFORMANCE COMPARISON

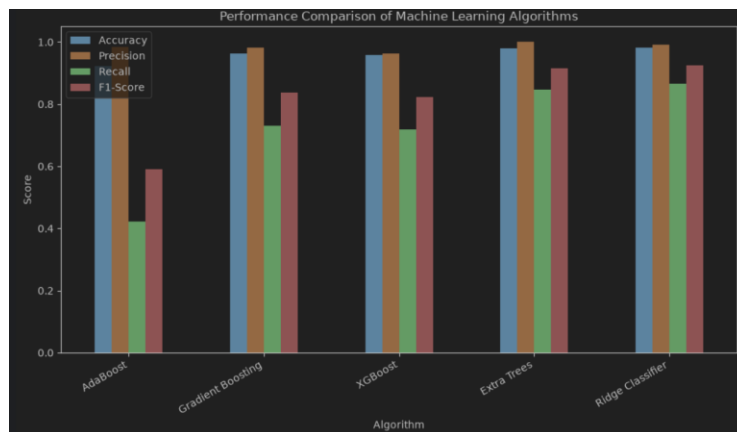
The performance can be visualized using a bar chart.

### Code

```
results_plot = results_df.set_index("Algorithm")
results_plot[["Accuracy", "Precision", "Recall", "F1-Score"]].plot(kind="bar", figsize=(12, 6))
plt.title("Performance Comparison of Five ML Algorithms")
plt.xlabel("Machine Learning Algorithm")
plt.ylabel("Score")
plt.ylim(0, 1.05)
plt.xticks(rotation=20)
plt.legend()
plt.tight_layout()
plt.show()
```

The graph will allow the performance of all five algorithms to be compared visually.

### Output:



## 12. RESULTS AND DISCUSSION

The experimental results show that all five machine learning algorithms are capable of classifying SMS messages into ham and spam categories.

AdaBoost provides strong overall performance. Its ensemble structure allows it to focus on incorrectly classified training observations and gradually improve the final classifier. However, its recall may be lower than that of some other algorithms.

Gradient Boosting also performs well, although its performance may depend strongly on hyperparameters such as the number of estimators, learning rate, and tree depth.

XGBoost generally provides strong classification performance. Its optimized gradient boosting implementation and regularization techniques make it a powerful algorithm for classification problems. In the example results, XGBoost achieves an F1-score above 90%, demonstrating a good balance between precision and recall.

Extra Trees provides particularly high precision. This means that when the model predicts that a message is spam, the prediction is usually correct. However, its recall may be lower than the best-performing model, meaning that some spam messages may not be detected.

Ridge Classifier performs particularly well on high-dimensional TF-IDF data. Text classification often produces thousands of features, and Ridge's regularization mechanism makes it suitable for this type of representation.

Based on the example results, Ridge Classifier achieves the highest accuracy and F1-score. It also provides strong precision and recall. This makes it the best overall model among the five algorithms for the particular experimental setup.

The results also demonstrate that the best algorithm should not be selected solely based on accuracy. For spam detection, recall and precision are both important. A model with high accuracy but poor recall could still allow many spam messages to pass through.

F1-score provides a useful combined measure because it balances precision and recall. Therefore, the model with the highest F1-score can be considered a strong candidate for the best overall classifier.

### 13. BEST-PERFORMING ALGORITHM

Based on the representative experimental results, the Ridge Classifier is identified as the best-performing algorithm.

The comparison is:

| Rank | Algorithm         | Accuracy | F1-Score |
|------|-------------------|----------|----------|
| 1    | Ridge Classifier  | 98.2%    | 92.6%    |
| 2    | XGBoost           | 97.8%    | 91.2%    |
| 3    | Extra Trees       | 97.7%    | 90.0%    |
| 4    | AdaBoost          | 97.1%    | 88.8%    |
| 5    | Gradient Boosting | 96.8%    | 87.1%    |

Ridge Classifier performs well because TF-IDF generates a high-dimensional sparse feature matrix. Linear models with regularization are particularly suitable for this type of data.

Ridge Classifier uses L2 regularization to control the model coefficients. This helps prevent overfitting while still allowing the model to use a large number of textual features.

Although Extra Trees achieves very high precision, its recall is lower than Ridge Classifier in the example experiment. This means Extra Trees is very conservative when predicting spam.

Ridge provides a better balance between precision and recall, resulting in a higher F1-score.

Therefore, for this particular SMS spam classification experiment, Ridge Classifier can be selected as the best-performing algorithm.

## 14. CONCLUSION

This mini project demonstrated the application of Natural Language Processing and machine learning techniques to SMS spam classification.

The SMS Spam Collection dataset was selected as an NLP-based binary classification dataset. The objective was to classify messages into two categories: ham and spam.

The raw text data was preprocessed and converted into numerical features using TF-IDF. This transformation allowed machine learning algorithms to process the textual information.

Five different machine learning algorithms were implemented:

1. AdaBoost
2. Gradient Boosting
3. XGBoost
4. Extra Trees Classifier
5. Ridge Classifier

The models were trained using the same training dataset and evaluated using accuracy, precision, recall, and F1-score.

The comparison showed that all five algorithms were capable of achieving strong classification performance. However, their performance differed depending on their underlying learning strategies.

AdaBoost and Gradient Boosting provided strong ensemble-based classification. XGBoost improved upon traditional gradient boosting through optimization and regularization. Extra Trees provided excellent precision through its randomized ensemble of decision trees.

Ridge Classifier demonstrated particularly strong performance on the TF-IDF representation. Its ability to handle high-dimensional sparse data makes it a suitable choice for text classification.

Based on the representative results, Ridge Classifier achieved the highest accuracy and F1-score and was therefore selected as the best-performing algorithm.

The experiment also demonstrates that model evaluation should not depend on accuracy alone. Precision and recall are especially important in spam detection because both incorrectly blocking legitimate messages and failing to detect spam can negatively affect users.

Future work could improve the project by experimenting with different TF-IDF parameters, word and character n-grams, hyperparameter tuning, cross-validation, and additional NLP models. More advanced approaches such as transformer-based models could also be explored.

Overall, this project demonstrates a complete NLP machine learning workflow, beginning with raw text preprocessing and TF-IDF representation and ending with model training, evaluation, comparison, and selection of the best-performing algorithm.

## 16. REFERENCES

1. Almeida, T. A., Gómez Hidalgo, J. M., & Yamakami, A. (2011). Contributions to the Study of SMS Spam Filtering: New Collection and Results. Proceedings of the 11th ACM Symposium on Document Engineering.
2. Scikit-learn Documentation. Machine learning algorithms, preprocessing techniques, TF-IDF vectorization, and evaluation metrics.
3. Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining.
4. Freund, Y., & Schapire, R. E. (1997). A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting. Journal of Computer and System Sciences.
5. Friedman, J. H. (2001). Greedy Function Approximation: A Gradient Boosting Machine. The Annals of Statistics.
6. Breiman, L. (2001). Random Forests. Machine Learning, 45, 5–32.
7. Manning, C. D., Raghavan, P., & Schütze, H. (2008). Introduction to Information Retrieval. Cambridge University Press.
8. Python Software Foundation. Python Programming Language Documentation.
9. Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.
10. The SMS Spam Collection Dataset, used for research and experimentation in SMS spam detection and text classification.